

Heuristic Optimization Techniques

Report 2

Aksinia Vorobeva, 12044614
Marcel Korinek, 12220824

January 17, 2026

1 Problem Description

Selective Capacitated Fair Pickup and Delivery Problem (SCF-PDP)

Graph model: Complete directed graph $G = (V, A)$, where:

- V contains the depot as well as all pickup and drop-off locations.
- $A = \{(u, v) : u, v \in V, u \neq v\}$ represents travel arcs.
- Each arc (u, v) has an associated distance

$$a_{u,v} = \left\lceil \sqrt{(x_u - x_v)^2 + (y_u - y_v)^2} \right\rceil,$$

where (x_u, y_u) are the coordinates of location u .

n Customer requests: For each request $i \in \{1, \dots, n\}$ there are:

$$v_i^\uparrow \in V \quad (\text{pickup location}), \quad v_i^\downarrow \in V \quad (\text{drop-off location}).$$

The solution must serve at least $\gamma \leq n$ of them.

Vehicles: A fleet K of n_K identical vehicles is available, each with capacity C and starting/ending at the depot.

Feasible solution: A solution consists of a route R_k for each vehicle $k \in K$, where $R_{k,i}$ denotes the i -th stop. Each solution must satisfy:

- Vehicle load never exceeds capacity C
- Each served request is fully handled by a single vehicle
- At least γ requests are served in total

The travel duration of route R_k is

$$d(R_k) = a_{\text{depot}, R_{k,1}} + a_{R_{k,|R_k|}, \text{depot}} + \sum_{(u,v) \in R_k} a_{u,v}.$$

Fairness measure: We use the Jain fairness index:

$$J(R) = \frac{(\sum_{k \in K} d(R_k))^2}{n_K \cdot \sum_{k \in K} d(R_k)^2}.$$

Objective: Minimize the total travel duration penalized by unfairness:

$$\sum_{k \in K} d(R_k) + \rho \cdot (1 - J(R)),$$

where ρ controls the trade-off between efficiency and fairness.

1.1 Experimental Setup

The experiments of instances with 50 customers for SA algorithm with different cooling strategies we made on machine with a AMD Ryzen 5 3500U and 8 RAM, running Windows 10 and Python 3.13.9.

All other experiments were made on a machine with a AMD Ryzen 7 9800X3D and 32 RAM, running Windows 11 and Python 3.13.9.

We tested every algorithm on all of the instances with size 50 and 100. For every non-deterministic algorithm we made at least three runs in order to increase the significance of the results.

For each experiment, the relevant algorithm configurations are provided in the corresponding chapter.

2 Description of Algorithms

Every algorithm shares five input parameters: *customers*, *vehicles*, *to_fulfilled*, *rho* and *output_statistic*.

Customers.

The input parameter *customers* is a list of **Customer** objects, each customer contains the relevant information for each request:

- a unique identifier
- a pickup location and a drop-off location, both represented as **Point** objects
- the amount of goods to be transported

A **Point** object contains the coordinates, the point type (depot, pickup, drop-off), and the quantity of goods associated with it (if the point type is pickup, then the quantity of goods is positive for pickups, if it is drop-off, then the quantity is negative).

Vehicles.

The most important parameter is *vehicles*, which is also our choice of a solution representation for this problem. It is a list of **Vehicle** objects. Each vehicle maintains the following:

- its capacity
- its current load and position
- its route, stored as a sequence of **Point** objects
- its accumulated path length
- a load-history tracing the load after each visited point

Together, the set of vehicles encodes the full multi-vehicle our routing solution, including which requests are served and in what order.

Minimum Requests to Serve.

The parameter *to_fulfill* describes the minimum number of requests to be served.

Fairness Weight ρ .

The parameter ρ is the fairness weight in the objective function. It is passed to the `ObjectiveTracker`, which evaluates

$$\text{Objective} = \sum_k d(R_k) + \rho \cdot (1 - J(R)),$$

and updates values efficiently using delta evaluation.

Output Statistics.

The parameter `output_statistic` is irrelevant for the functionality of the algorithms, as it is only relevant for the experimental evaluation and result analysis. We save the collection of detailed performance data using the `Statistic` class, including:

- the objective value over time
- fairness values over time
- total route duration over time
- the number of iterations performed

Algorithm output.

Ultimately, at the end of the algorithm, we receive:

- a file in .txt format with the final solution to the problem
- statistical data
- a graphical representation of the solution (see Figure 1)

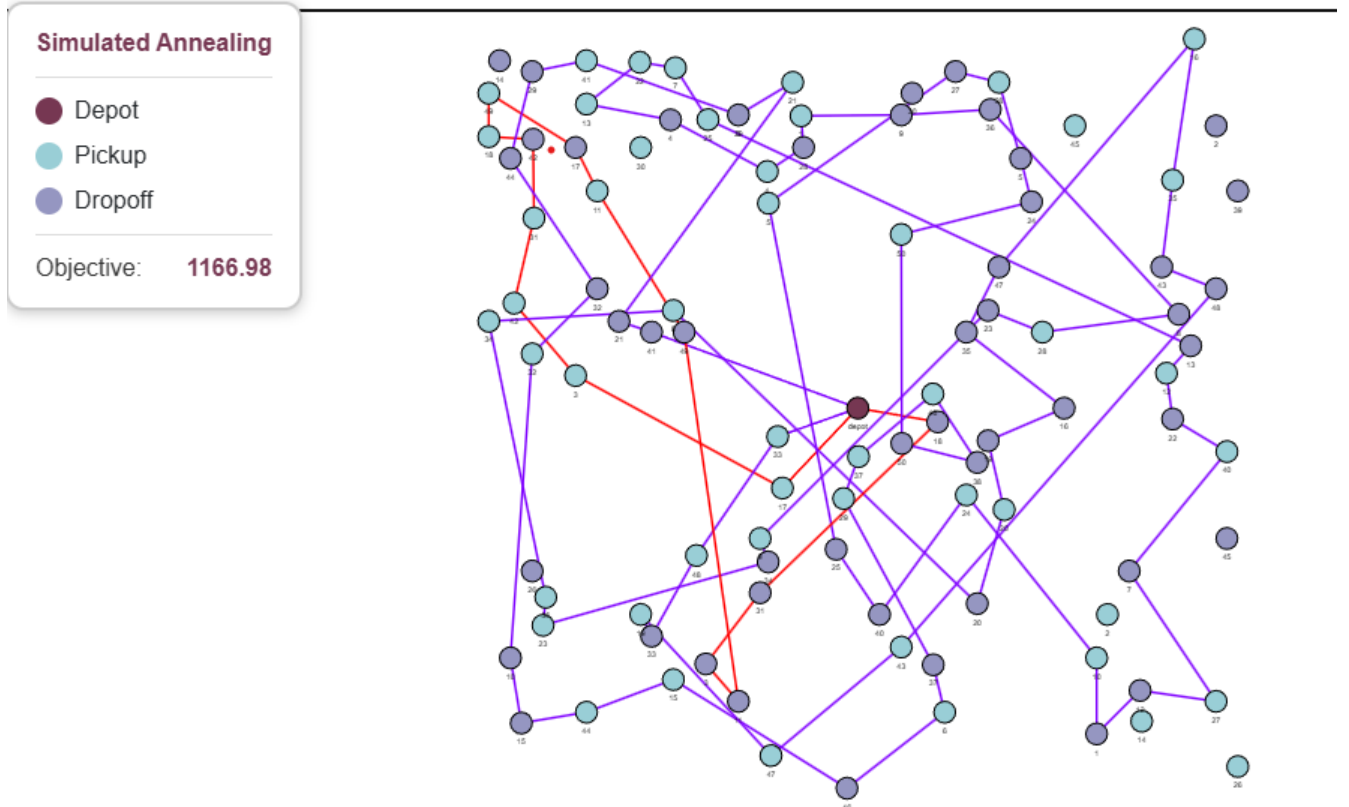


Figure 1: Graphical representation of SA output

2.1 Genetic Algorithm

The proposed solution method is based on a Genetic Algorithm (GA), an evolutionary meta-heuristic that iteratively improves a population of candidate solutions through biologically inspired operators.

The Genetic Algorithm is controlled by a set of problem-specific and algorithmic parameters, summarized below.

- **customers**: List of all `Customer` objects.
- **vehicles**: List of a given number of empty `Vehicle` objects.
- **to_fulfilled**: Minimum number of customer requests that must be fulfilled.
- **rho**: Fairness weight.
- **s**: Selection pressure.
- **t_max**: Maximum number of generations (iterations).
- **population_size**: Number of individuals in the population.
- **num_elites**: Number of elite individuals preserved during replacement.
- **selection_method**: Parent selection strategy.
- **tournament_size**: Number of individuals competing in each tournament (only relevant if tournament selection is the chosen selection method).
- **tournament_replace**: Sets whether an individual can be chosen multiple times for a tournament (only relevant if tournament selection is the chosen selection method).
- **recombination_rate**: Probability of applying crossover to a selected pair.
- **recombination_weights**: Operator-specific weights.
- **recombination_samples**: Number of tries to insert a pickup point and a dropoff point randomly in a vehicle path (relevant for submethods of the recombination)
- **mutation_rate**: Probability of mutating an offspring.
- **mutation_weights**: Operator-specific weights.
- **output_statistic**: Boolean value indicating if the more details regarding the run should be returned.

The overall procedure consists of the following main phases:

1. **Initialization:** Generation of an initial population of feasible solutions.
2. **Evaluation:** Computation of the objective value (fitness) for each individual.
3. **Selection:** Choice of parent individuals for Recombination based on their fitness.
4. **Recombination:** Creation of new offsprings by combining solution parts from selected parents.
5. **Mutation:** Random modification of offsprings to maintain diversity and explore new regions of the search space.
6. **Replacement:** Construction of the next generation by replacing the old population by the offsprings while potentially also preserving elite individuals.

Steps 2 to 6 are repeated until a maximum number of iterations is reached or the algorithm exceeds a runtime of 10 minutes.

Each of the phases listed above is described in detail in the subsequent chapters.

2.1.1 Initialization

Each individual of the initial population is generated using a fast constructive procedure. For every customer request, a vehicle is selected uniformly at random, and the corresponding pickup and drop-off nodes are appended to the current route of that vehicle. This procedure is computationally efficient and yields a roughly balanced initial assignment of customers across all vehicles. The resulting solutions are not necessarily of high quality. However, this is acceptable, as the primary objective of the initialization phase is to provide diverse and feasible starting points for the evolutionary search rather than near-optimal solutions.

2.1.2 Evaluation

The evaluation phase assigns a fitness value to each individual in the population and consists of two main steps.

Fitness Transformation. Since the considered problem is a minimization problem, while the Genetic Algorithm operates on a maximization of fitness, the objective values are first transformed accordingly. Let c_i denote the objective value of individual i and let

$$c_{\max} = \max_{j \in P} c_j + 1$$

be a constant larger than any objective value in the current population P . The raw fitness of individual i is then computed as

$$f_i = c_{\max} - c_i.$$

By construction, all fitness values are strictly positive and a smaller objective value results in a larger fitness.

Linear Scaling. To control the selection pressure and avoid premature convergence, a linear scaling of the raw fitness values is applied. Let $f_{\max}$, f_{avg} , and $f_{\min}$ denote the maximum, average, and minimum fitness in the population, respectively. The scaled fitness $\hat{f}_i$ is obtained as

$$\hat{f}_i = af_i + b,$$

where the scaling coefficients a and b are chosen such that

$$\hat{f}_{\text{avg}} = f_{\text{avg}}, \quad \hat{f}_{\max} = s \cdot \hat{f}_{\text{avg}},$$

with $s > 1$ being a user-defined scaling factor. Solving this system yields

$$a = \frac{(s-1)\hat{f}_{\text{avg}}}{f_{\max} - \hat{f}_{\text{avg}}}, \quad b = f_{\text{avg}} - a\hat{f}_{\text{avg}}.$$

In order to guarantee non-negative scaled fitness values, the coefficients are further adjusted if $\hat{f}_{\min} < 0$ by enforcing $\hat{f}_{\min} = 0$. In this case, the parameters are recomputed as

$$a = \frac{\hat{f}_{\text{avg}}}{\hat{f}_{\text{avg}} - f_{\min}}, \quad b = a\hat{f}_{\text{avg}} - \hat{f}_{\text{avg}},$$

The resulting scaled fitness values are finally used in the subsequent selection phase. This linear scaling mechanism preserves the ranking of individuals while regulating the difference between the best and average fitness, thereby balancing exploration and exploitation during the evolutionary search.

Right after the evaluation phase, the solution of the best individual of the recently evaluated population is compared to the best found solution so far. This ensures that at the end of the whole algorithm always the best solution of all generations is returned.

2.1.3 Selection

The selection phase determines which individuals of the current population are chosen as parents for the recombination. Individuals with higher fitness values are given a higher probability of being selected, thereby implementing the principle of survival of the fittest while still preserving stochasticity to maintain population diversity. If a number of elites is specified, this will be considered by choosing only `population_size - num_elites` parents. Two selection mechanisms are implemented: roulette-wheel selection and tournament selection.

Roulette-Wheel Selection. In roulette-wheel (fitness-proportionate) selection, each individual i is assigned a selection probability proportional to its fitness value f_i . Let P be the current population and $F = \sum_{j \in P} f_j$ the sum of all fitness values. The probability of selecting individual i is given by

$$p_i = \frac{f_i}{F}.$$

Parents are then sampled from the population according to this discrete probability distribution. As a consequence, individuals with higher fitness are more likely to be selected, while lower-quality individuals still retain a non-zero chance of contributing to the next generation.

Tournament Selection. In tournament selection, a set of k individuals (the tournament size) is drawn uniformly at random from the population, either with or without allowing duplicates (toggled by the replacement parameter). The individual with the highest fitness among the selected contenders is declared the winner of the tournament and is chosen as a parent. This procedure is repeated until the required number of parents has been selected.

The tournament size k directly controls the selection pressure: larger values of k increase the probability that highly fit individuals are selected, while smaller values promote diversity.

Both selection strategies preserve the ranking induced by the fitness values, but differ in their stochastic behavior and selection pressure. The choice between them and the tuning of their parameters significantly influence the balance between exploration and exploitation in the evolutionary process.

2.1.4 Recombination

The recombination phase generates new candidate solutions (offsprings) by combining structural components of two parent individuals. Let Q_s denote the list of selected parents. Parents are processed sequentially in pairs. For each pair (p_1, p_2) , recombination is applied with probability `recombination_rate`. If recombination is not applied, both parents are copied unchanged to the offspring population.

If recombination is triggered, one recombination operator is chosen randomly from a predefined set of operators, where the probability of choosing a specific operator is controlled by the user-defined weights `recombination_weights`. Depending on the chosen operator, either one or two children may be produced. If only one child is returned, the same operator is applied again with reversed parent roles to obtain a second offspring. If the number of selected parents is odd, the last unpaired parent is copied directly into the offspring population.

Customer-Based Recombination. In customer-based recombination, a child is initialized as a copy of the first parent. Then, for each customer, a decision is made independently with a fixed probability (set to 0.8) whether the customer should be kept from the first parent. All customers that are not selected to be kept are removed from the child solution by deleting both their pickup and drop-off nodes from the corresponding vehicle route.

The missing customers are reinserted according to the order induced by the second parent. For each missing customer, the algorithm first checks in which vehicle the customer appears in the second parent. If such a vehicle exists, the customer is inserted into the corresponding vehicle of the child; otherwise, the customer is assigned to the currently shortest vehicle route. In both cases, insertion is performed by a stochastic insertion heuristic that tries several random pickup and drop-off positions and keeps the most promising candidates with respect to the objective function.

After all missing customers have been reinserted, a repair procedure is applied to eliminate duplicates, resolve infeasibilities, and enforce the minimum number of fulfilled requests. This operator produces exactly one child.

Vehicle-Based Recombination. In vehicle-based recombination, two children are created simultaneously. First, copies of both parents are made. The vehicle list of the first child is randomly permuted. Then, for each vehicle position k , the vehicles of the two children are swapped with probability 0.5. This produces offsprings that inherit entire vehicle routes from both parents.

Since this operation may create duplicate or missing customer assignments, a repair procedure is subsequently applied to both children to restore feasibility and to ensure that each customer appears at most once and that at least `to_fulfilled` requests are served.

Stochastic Insertion. Both recombination operators rely on a stochastic insertion subroutine for placing a customer into a given vehicle route. For a customer with pickup node p and drop-off node d , random pairs of insertion positions (i, j) with $i < j$ are sampled, where p is inserted after position i and d after position j . For each sampled pair, the resulting path length and its impact on the global objective function are estimated. Among all tried candidates, the best `samples` insertion positions are kept.

These candidates are then tested in increasing order of predicted objective value. The first insertion that yields a valid route is accepted. If none of the candidates is feasible, a fallback strategy inserts the pickup just before the final depot and the drop-off immediately after the pickup, followed by a reordering step to restore consistency.

Repair Procedure. After recombination, offspring solutions may contain customers multiple times or may violate feasibility conditions. The repair procedure first scans all customers and removes duplicates by keeping the occurrence in the shorter vehicle route and deleting the other.

If the solution already satisfies the minimum number of fulfilled requests, it is accepted. Otherwise, all currently unassigned customers are collected. While the solution is infeasible and fewer than `to_fulfilled` insertion attempts have been made, customers are selected randomly from this set and inserted into the currently shortest route using the stochastic insertion procedure.

If no unassigned customers remain but the solution is still infeasible, the entire child is reset to a copy of the first parent, ensuring that recombination never produces an unusable individual.

2.1.5 Mutation

The mutation phase introduces random local changes into offspring solutions in order to preserve diversity and to reduce the risk of premature convergence. Let Q_r denote the population after recombination. Each individual is mutated independently with probability `mutation_rate`. If an individual is selected for mutation, one mutation operator is drawn randomly from a predefined set, where the probability of choosing a specific operator is controlled by the user-defined weights `mutation_weights`. If no mutation is applied, the individual is passed unchanged to the next population.

Swap Mutation. Swap mutation modifies the assignment of two randomly chosen customer requests. Two distinct customers are sampled uniformly at random. Let v_1 and v_2 denote the vehicles currently serving the first and second customer, respectively. Three cases are distinguished:

- If both customers are unassigned, the attempt is discarded.
- If exactly one customer is assigned to a vehicle, that customer is removed from its vehicle and replaced by the unassigned customer, inserting the pickup and drop-off nodes of the previously unassigned customer at the same positions.
- If both customers are assigned, their pickup and drop-off nodes are exchanged between the two vehicles.

After performing the exchange or replacement, all affected vehicle routes are checked for feasibility. If the modified routes are valid, the mutated individual is accepted. Otherwise, the attempt is rejected and repeated. This process is carried out up to a fixed number of attempts, after which the original individual is returned if no valid mutation could be generated.

Move Mutation. Move mutation relocates a single customer from its current vehicle to a different vehicle. First, a customer that is currently assigned to some vehicle is selected uniformly at random. Its current vehicle is identified, and another vehicle is chosen uniformly at random as the target.

Random insertion positions for the pickup and drop-off nodes are selected in the target vehicle route, excluding the depot positions. The customer is removed from its original vehicle and inserted into the target vehicle at the chosen positions. If both modified vehicle routes are feasible after the move, the mutation is accepted. Otherwise, the attempt is rejected and repeated. As for swap mutation, only a limited number of trials is performed before reverting to the original individual.

Neighborhood-Based Mutation. Neighborhood-based mutation applies a local improvement step rather than a purely random change. One neighborhood structure is selected uniformly at random from a predefined set, including exchange, pickup relocation, drop-off relocation, remove-and-append, and move operations. Starting from the current solution, a neighboring solution is generated according to the chosen structure using a first-improvement strategy guided by the objective function and the fairness parameter ρ .

If a strictly better neighboring solution is found, the individual is replaced by this improved solution. Otherwise, the individual remains unchanged. This operator therefore acts as an intensification mechanism that exploits the local structure of the search space.

2.1.6 Replacement

The replacement phase constructs the population of the next generation by combining elite individuals (if there are any) from the current population with the newly generated offsprings. This preserves the size of the population throughout every iteration.

After the replacement, the new population continues with the evaluation phase.

2.1.7 Parameter Tuning

The performance of a Genetic Algorithm strongly depends on the choice of its parameters. Therefore, a systematic parameter tuning process was carried out using the automatic configuration tool *irace*. Tuning was performed separately for problem instances of size 50 and 100 from the set of training instances, as these instance classes differ significantly in computational difficulty and runtime behavior.

Tuning Setup. For the instance set of size 50, the tuning scenario was defined with a maximum of 300 experiments and a confidence level of 0.95. The tuning process executed the algorithm through a batch script, which forwarded the parameter values proposed by *irace* to the main solver via the command line.

Each run of the algorithm returned only the final objective value, which was used by *irace* as the performance measure to be minimized.

Tuned Parameters. The following parameters were included in the tuning process:

- Number of generations $t_{\max}$
- Population size
- Selection pressure s
- Selection method (roulette-wheel or tournament)
- Tournament size and replacement policy
- Recombination operator weights
- Mutation operator weights
- Number of elite individuals
- Number of recombination samples
- Recombination rate
- Mutation rate

Instance-Specific Tuning. Tuning was carried out independently for instances of size 50 and 100. This separation was necessary because larger instances impose stricter time limits, which changes the trade-off between population size, number of generations, and operator complexity. By tuning both instance classes separately, parameter configurations could be adapted to the different computational budgets and problem characteristics.

Tuned Parameter Configurations. These resulting configurations, shown below for each instance size, were then fixed and used for all subsequent experimental evaluations. This ensures that the reported results reflect well-calibrated algorithmic behavior rather than arbitrary parameter choices.

Table 1: Resulting GA parameter configurations for instances of size 50 and 100.

Parameter	Size 50	Size 100
Number of generations, $t_{\max}$	170	134
Population size	62	76
Selection pressure, s	1.0625	1.5719
Selection method	tournament	tournament
Tournament size	62	35
Tournament replace	False	False
Recombination weights	[0.7411, 0.1901]	[0.7883, 0.4033]
Mutation weights	[0.3749, 0.3587, 0.4831]	[0.7198, 0.5541, 0.537]
Number of elite individuals	24	32
Recombination samples	176	10
Recombination rate	0.4257	0.9743
Mutation rate	0.1647	0.4273

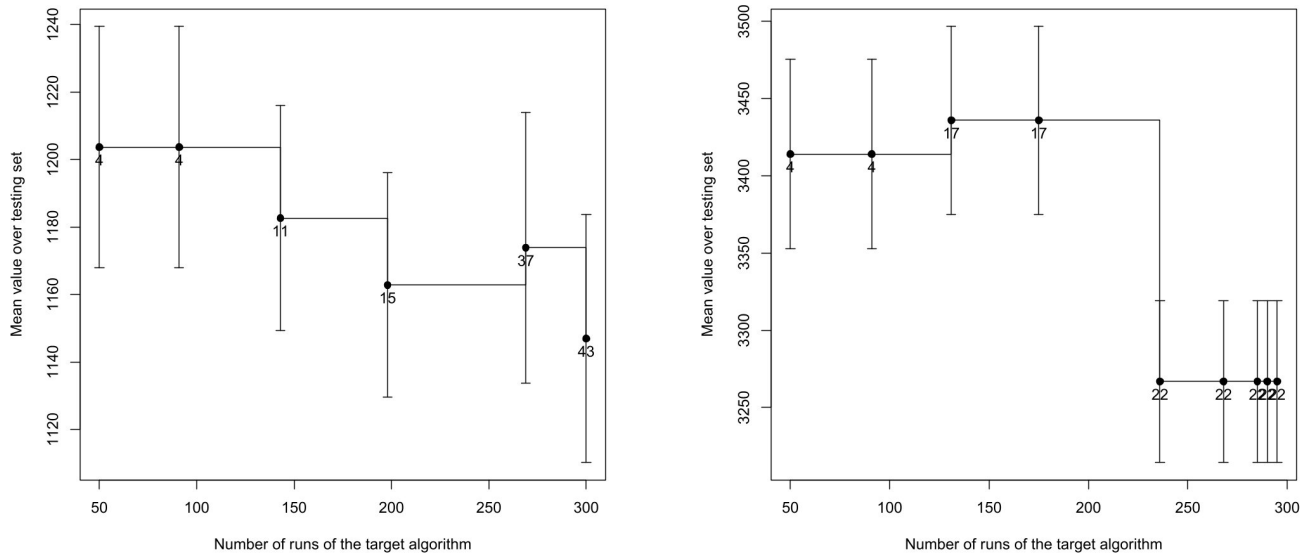


Figure 2: Configuration Quality Over Time for Instances of Size 50 (left) and 100 (right).

Figure 2 shows how the quality of the best parameter configuration improved as the number of tuning runs increased.

2.1.8 Evolution of Solution Quality and Fairness

Figure 3 illustrates the improvement of the solution quality over time. The most significant progress is achieved during the initial iterations, after which the objective value change gets smaller and converges towards a stable value.

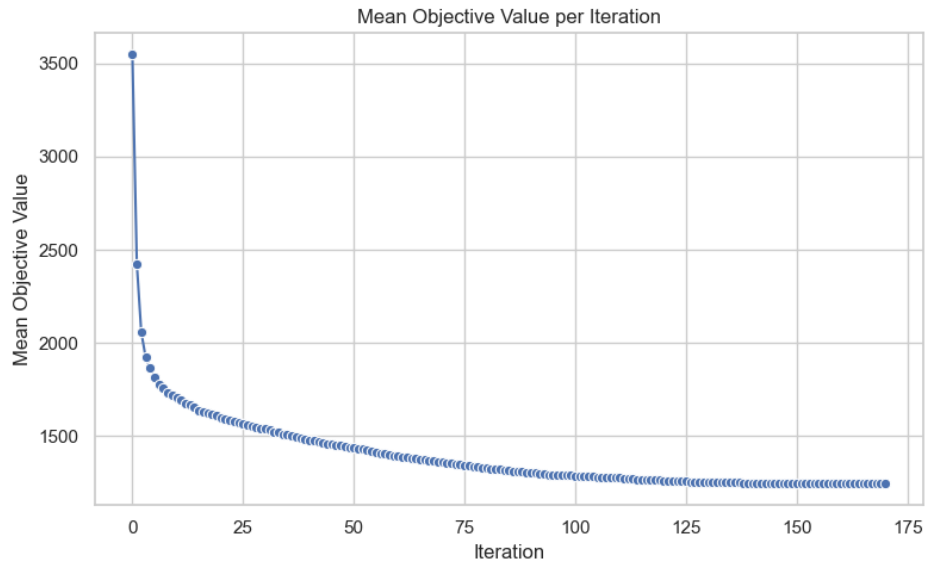


Figure 3: Mean Objective Value Over Iterations for 50 Customers (GA)

The evolution of fairness over iterations is presented in Figure 4. Although the fairness slightly decreases, it remains above 0.975, which is still a very high fairness value.

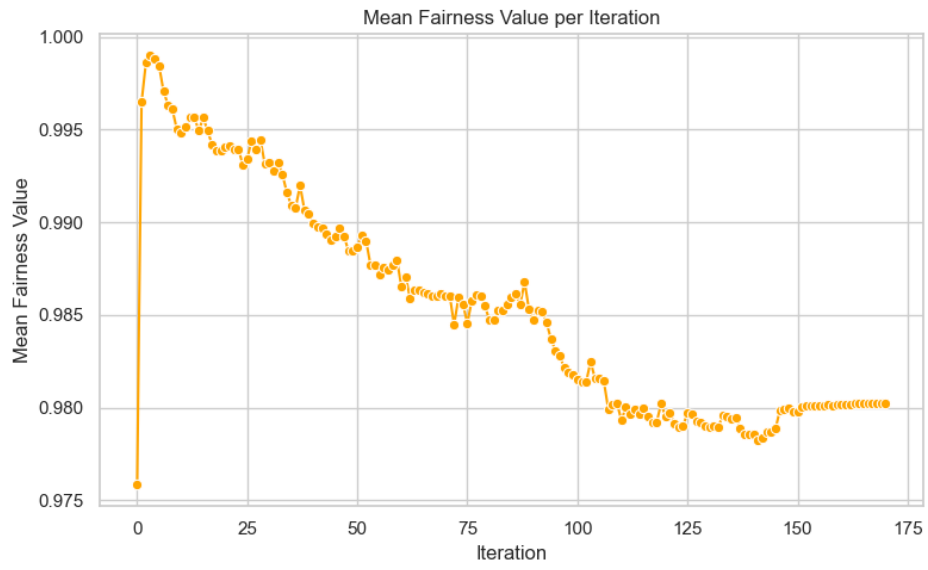


Figure 4: Mean Fairness Value Over Iterations for 50 Customers (GA)

2.2 Ant Colony Optimization

2.2.1 Evolution of Solution Quality and Fairness

Figure 3 shows how the solution quality improves throughout the iterations.

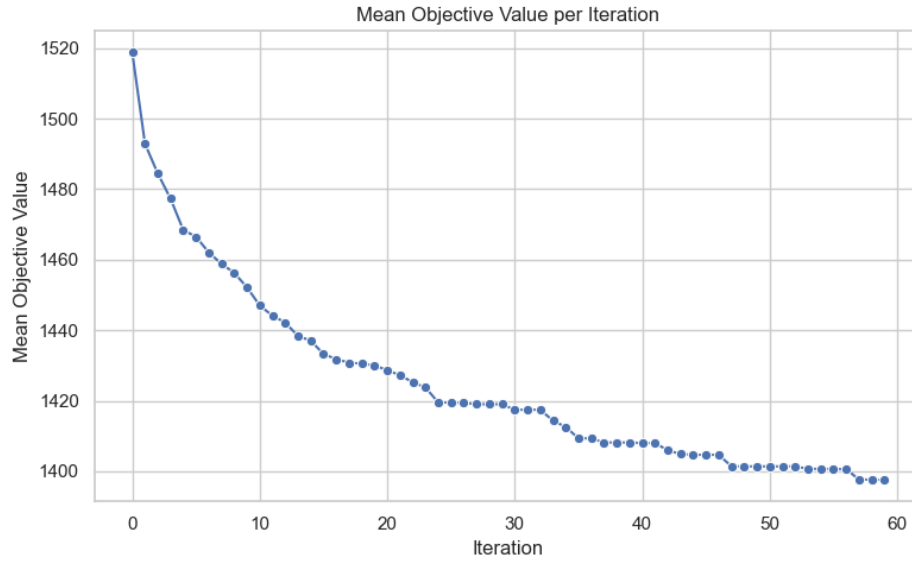


Figure 5: Mean Objective Value Over Iterations for 50 Customers (ACO)

Figure 4 illustrates the evolution of the fairness value over the course of the iterations. Overall, the fairness metric exhibits an increasing trend, rising from an initial value of approximately 0.89 to about 0.94.

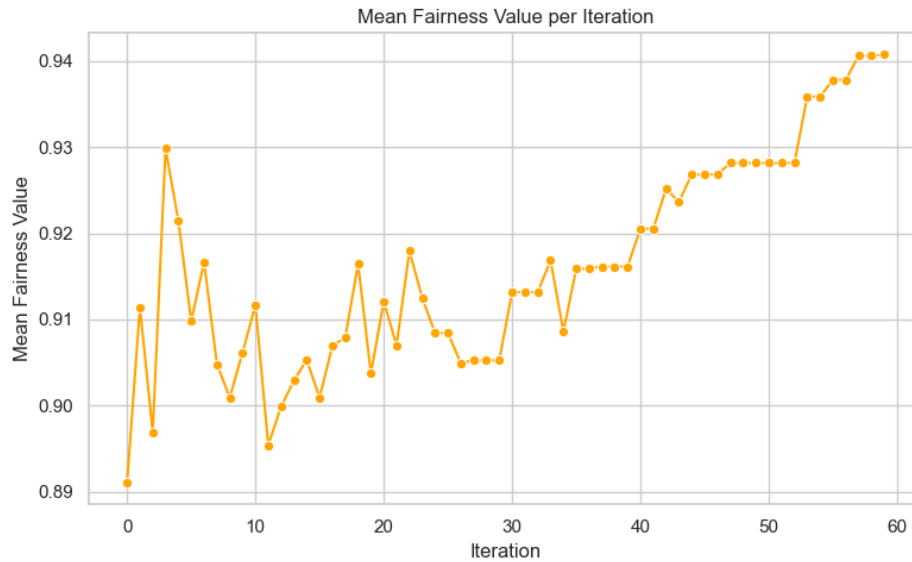


Figure 6: Mean Fairness Value Over Iterations for 50 Customers (ACO)

2.3 Comparison Between GA and ACO

This subsection analyzes the performance of GA and ACO.

2.3.1 Experimental Results

The experimental results show that GA consistently produces lower objective values than ACO for both instances of size 50 and 100. This behavior can be observed in the boxplots (Figures 7 and 8) and in the per-instance bar charts (Figures 9 and 10), where GA dominates ACO across all tested instances. The visual analysis already suggests a clear advantage of GA over ACO in terms of solution quality.

Regarding computational efficiency, ACO demonstrates a clear advantage over GA. As illustrated in the right parts of Figures 7 and 8, ACO consistently requires significantly less CPU time across all instances. While GA provides higher solution quality, the lower execution time of ACO suggests it may be more suitable for time-sensitive applications where a rapid solution is required.

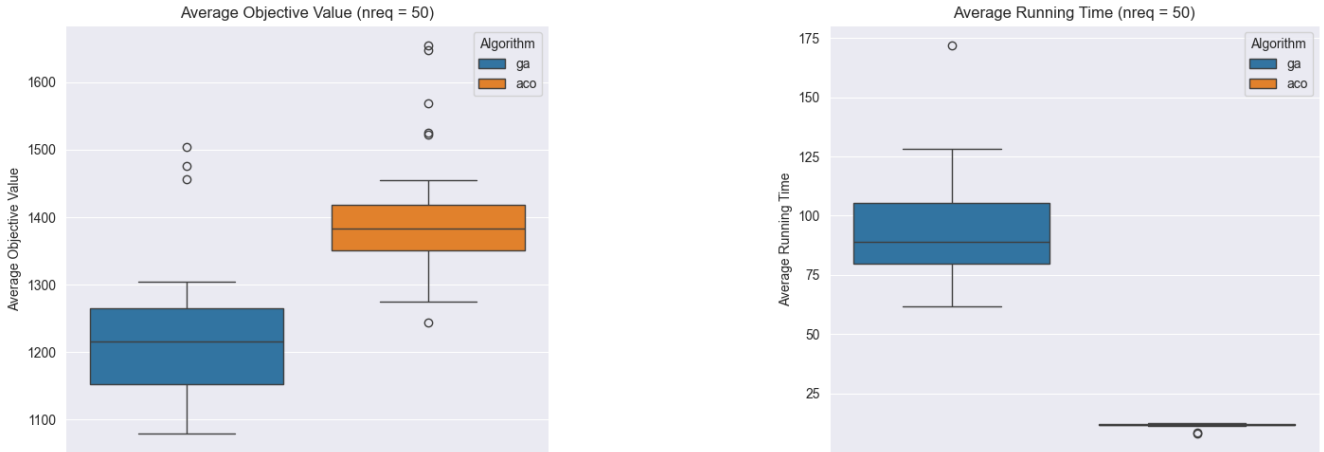


Figure 7: Comparison GA vs. ACO: Average Objective Function and Run Time for 50 Customers

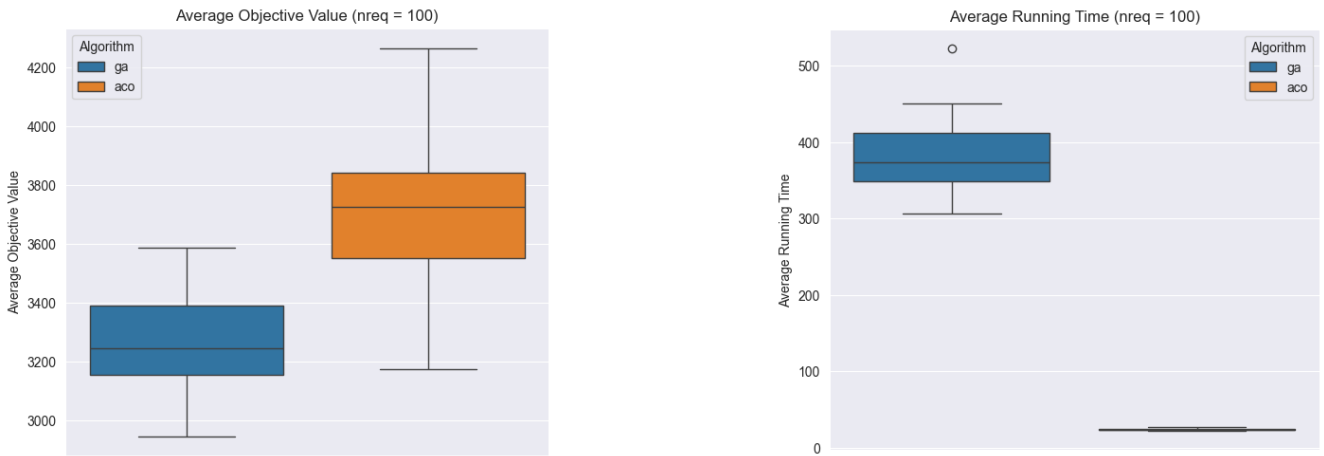


Figure 8: Comparison GA vs. ACO: Average Objective Function and Run Time for 100 Customers



Figure 9: Comparison GA vs. ACO: Average Objective Function and Run Time for 50 Customers per Instance

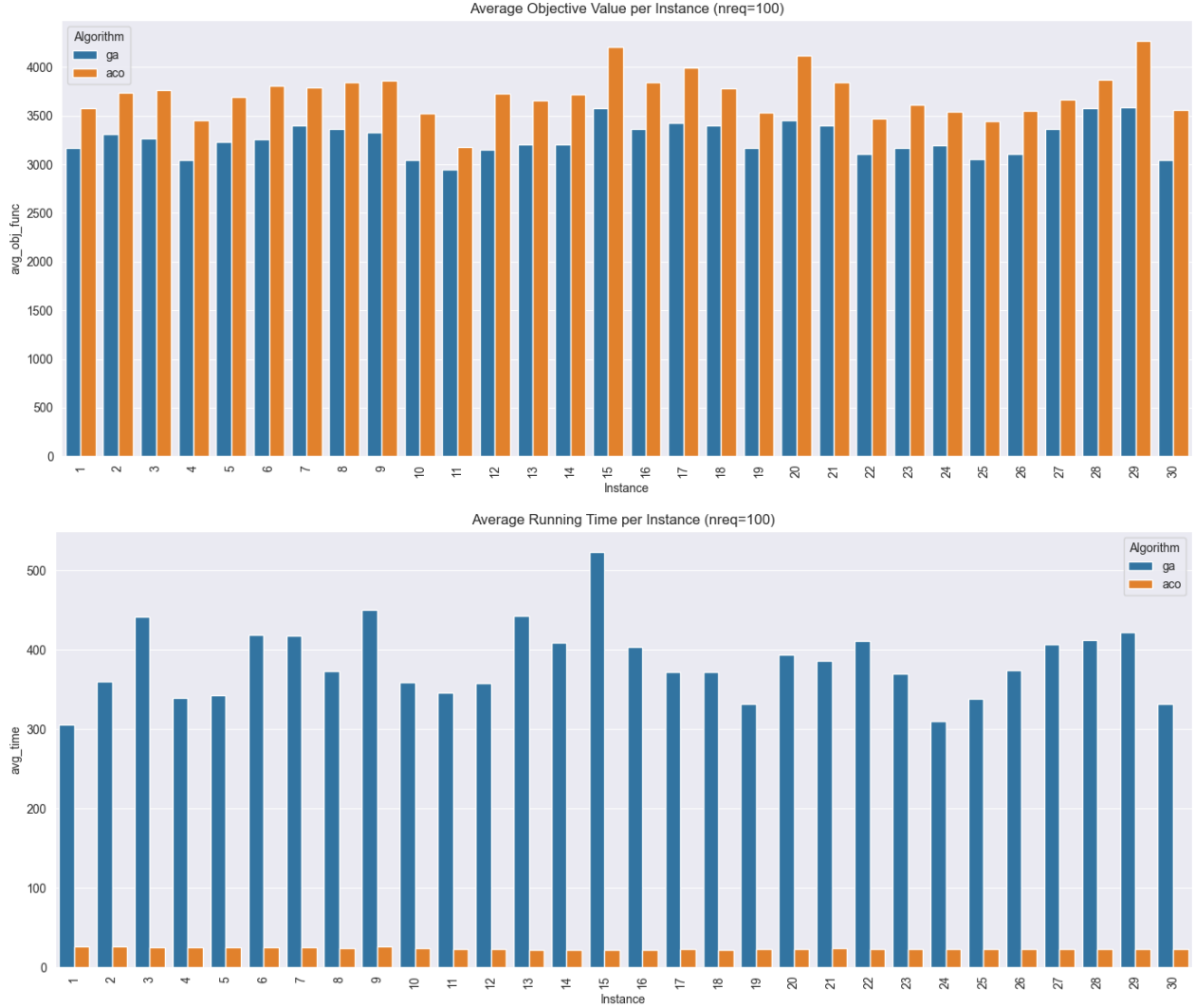


Figure 10: Comparison GA vs. ACO: Objective Function and Run Time for 100 Customers per Instance

2.3.2 Statistical Validation

To formally verify this observation, a one-sided Wilcoxon signed-rank test was applied using the following hypotheses:

$$H_0 : GA \geq ACO \quad \text{vs} \quad H_1 : GA < ACO.$$

Tests were conducted separately for $nreq = 50$ and $nreq = 100$ with a significance level of $\alpha = 0.05$.

Table 2: One-sided Wilcoxon test: GA better than ACO

$nreq$	W	p -value	Conclusion
50	0	9.31×10^{-10}	GA significantly better
100	0	9.31×10^{-10}	GA significantly better

For both problem sizes, the test statistic $W = 0$ indicates that GA achieves lower objective values than ACO on all paired instances. The extremely small p-values provide overwhelming statistical evidence that GA outperforms ACO in terms of solution quality.

2.4 Comparison Between GA and VND

After identifying GA as superior to ACO in terms of solution quality, GA is compared with VND, which was the best performing algorithm of the previous exercise.

2.4.1 Experimental Results

The experimental analysis shows that VND generally achieves lower objective values than GA for both $nreq = 50$ and $nreq = 100$. This trend is visible in the boxplots (Figures 11 and 12) and per-instance bar charts (Figures 13 and 14). For $nreq = 100$, VND dominates GA on almost all instances, while for $nreq = 50$ only a small number of instances favor GA.

For the smaller instances ($nreq = 50$), VND exhibits superior efficiency, converging significantly faster than GA. However, this trend reverses for the larger instances ($nreq = 100$). While the execution time of GA is artificially constrained by a fixed computational budget (a 10-minute time limit), VND's running time scales more aggressively with the problem size. Consequently, for $nreq = 100$, VND becomes the more computationally intensive approach, whereas GA remains bounded by the predefined time threshold. This suggests that while VND achieves higher solution quality, it does so at the cost of significantly higher temporal requirements in larger search spaces.

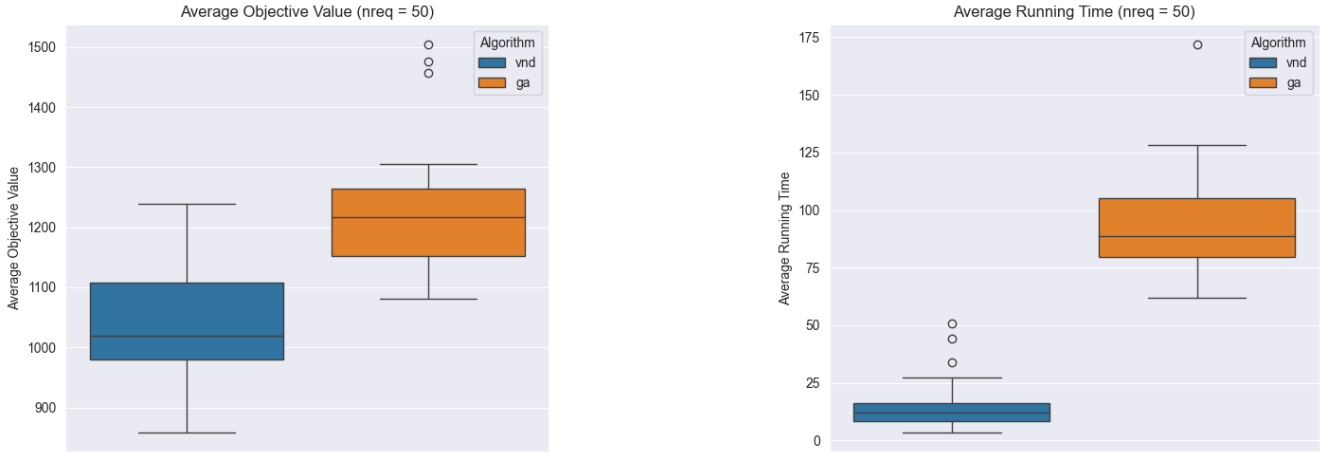


Figure 11: Comparison VND vs. GA: Average Objective Function and Run Time for 50 Customers

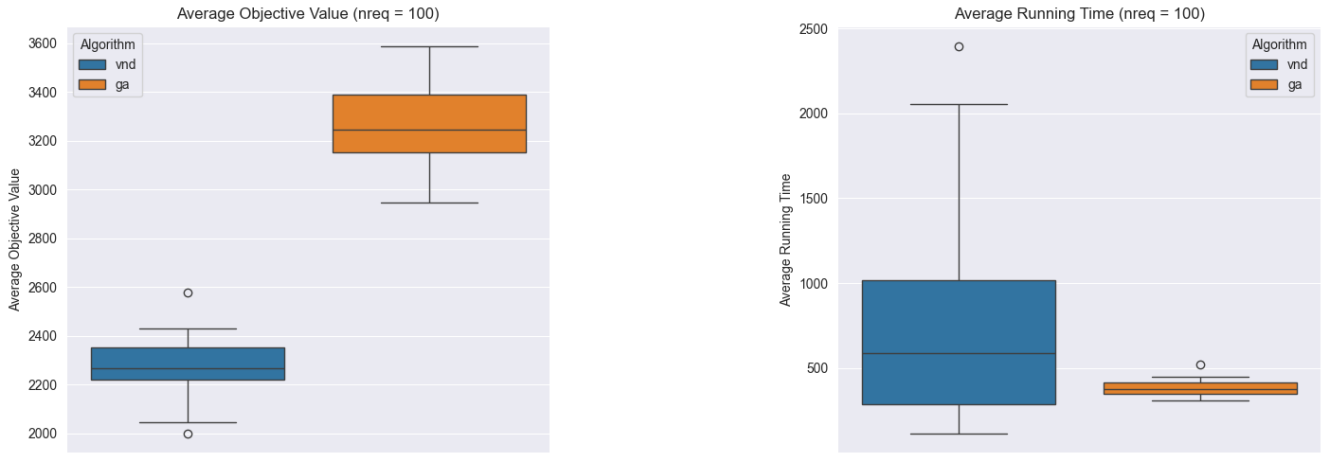


Figure 12: Comparison VND vs. GA: Average Objective Function and Run Time for 100 Customers

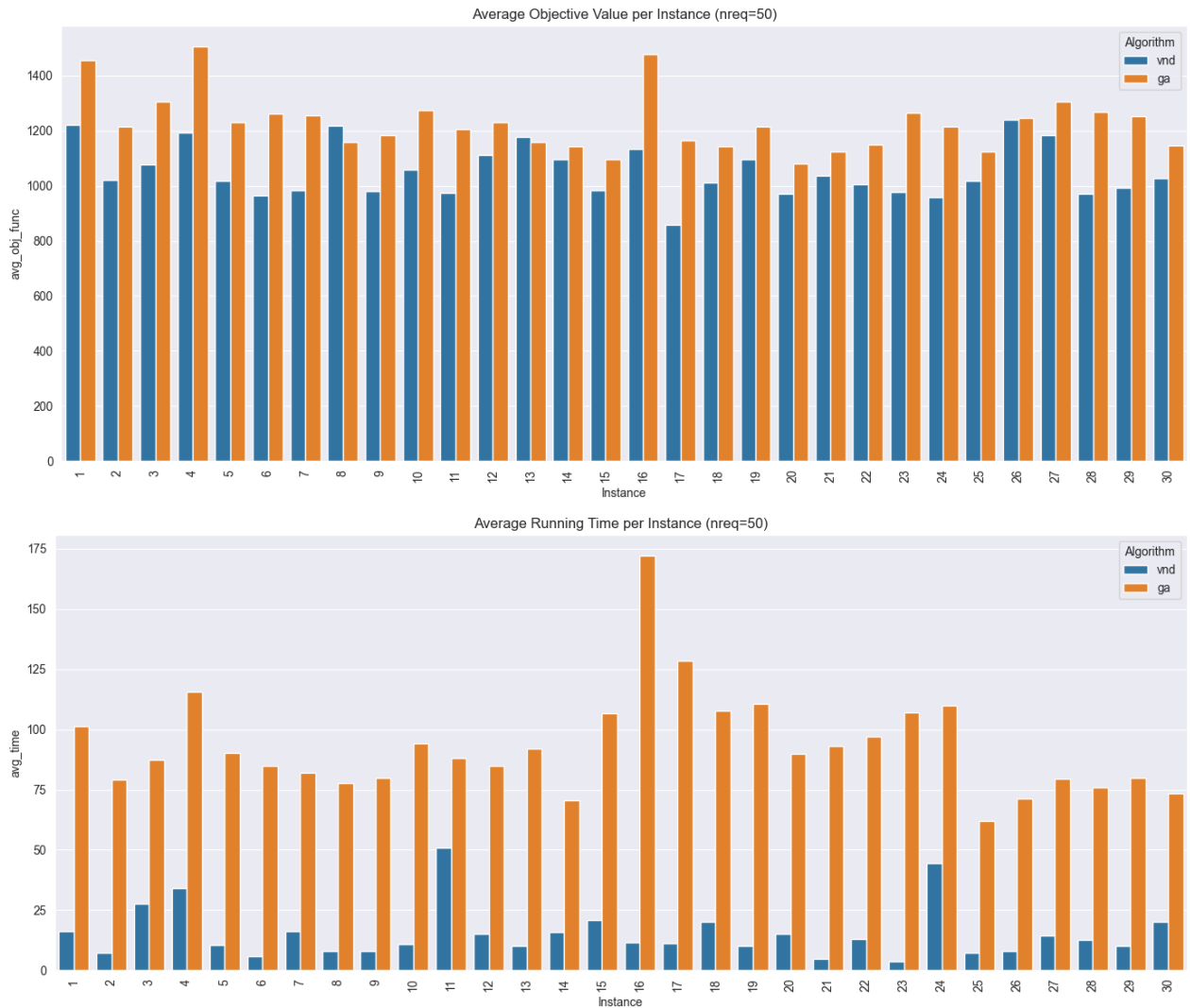


Figure 13: Comparison VND vs. GA: Average Objective Function and Run Time for 50 Customers per Instance

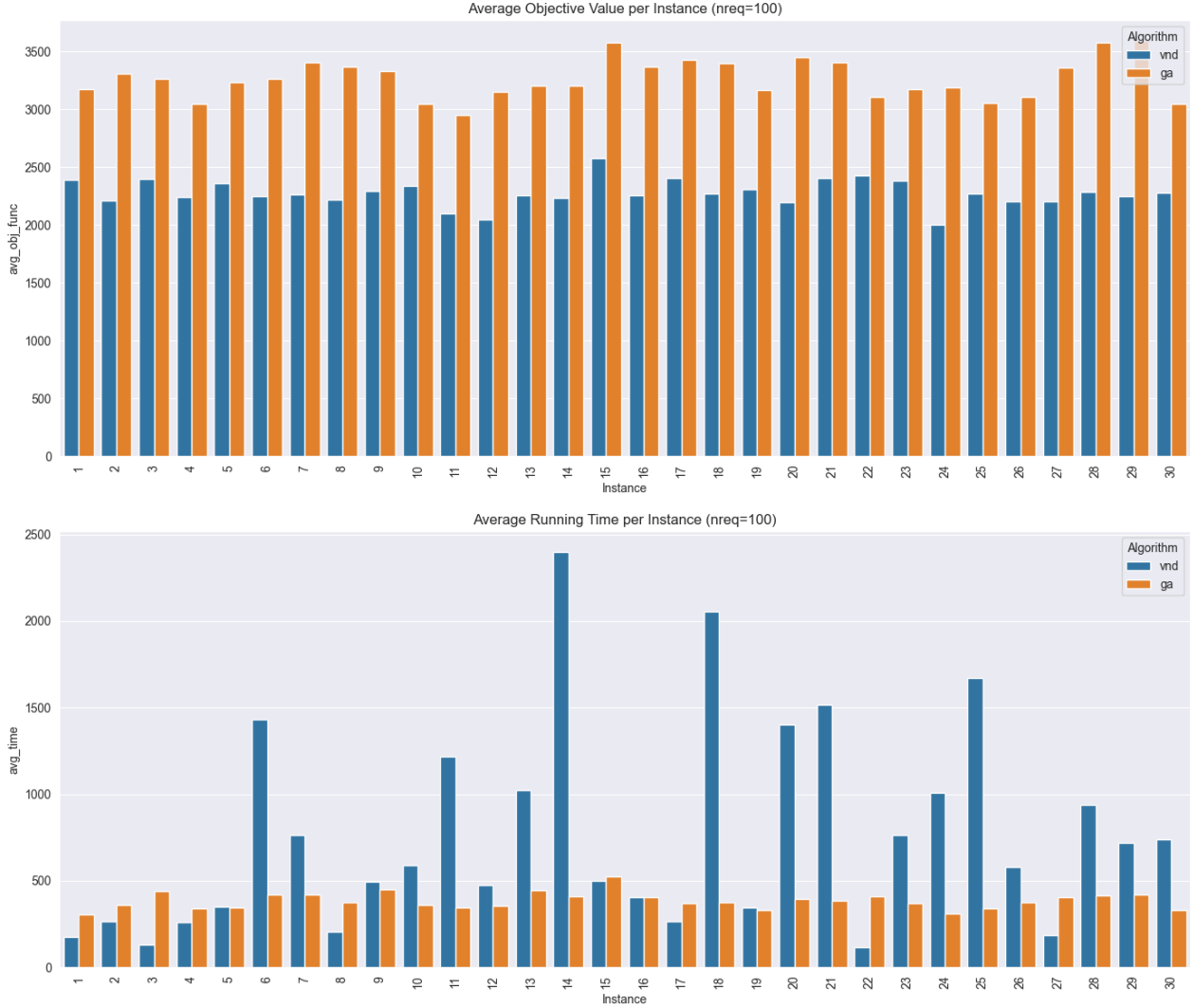


Figure 14: Comparison VND vs. GA: Objective Function and Run Time for 100 Customers per Instance

2.4.2 Statistical Validation

To validate these observations, a one-sided Wilcoxon signed-rank test was performed with the following hypotheses:

$$H_0 : \text{VND} \geq \text{GA} \quad \text{vs} \quad H_1 : \text{VND} < \text{GA}.$$

Again, tests were conducted for $nreq = 50$ and $nreq = 100$ using $\alpha = 0.05$.

Table 3: One-sided Wilcoxon test: VND better than GA

$nreq$	W	p -value	Conclusion
50	6	1.30×10^{-8}	VND significantly better
100	0	9.31×10^{-10}	VND significantly better

For $nreq = 50$, a small number of instances favor GA, resulting in $W = 6$, but the p -value remains extremely small. For $nreq = 100$, VND produces lower objective values than GA on all

instances ($W = 0$). In both cases, the one-sided Wilcoxon test confirms that VND significantly outperforms GA in terms of solution quality.